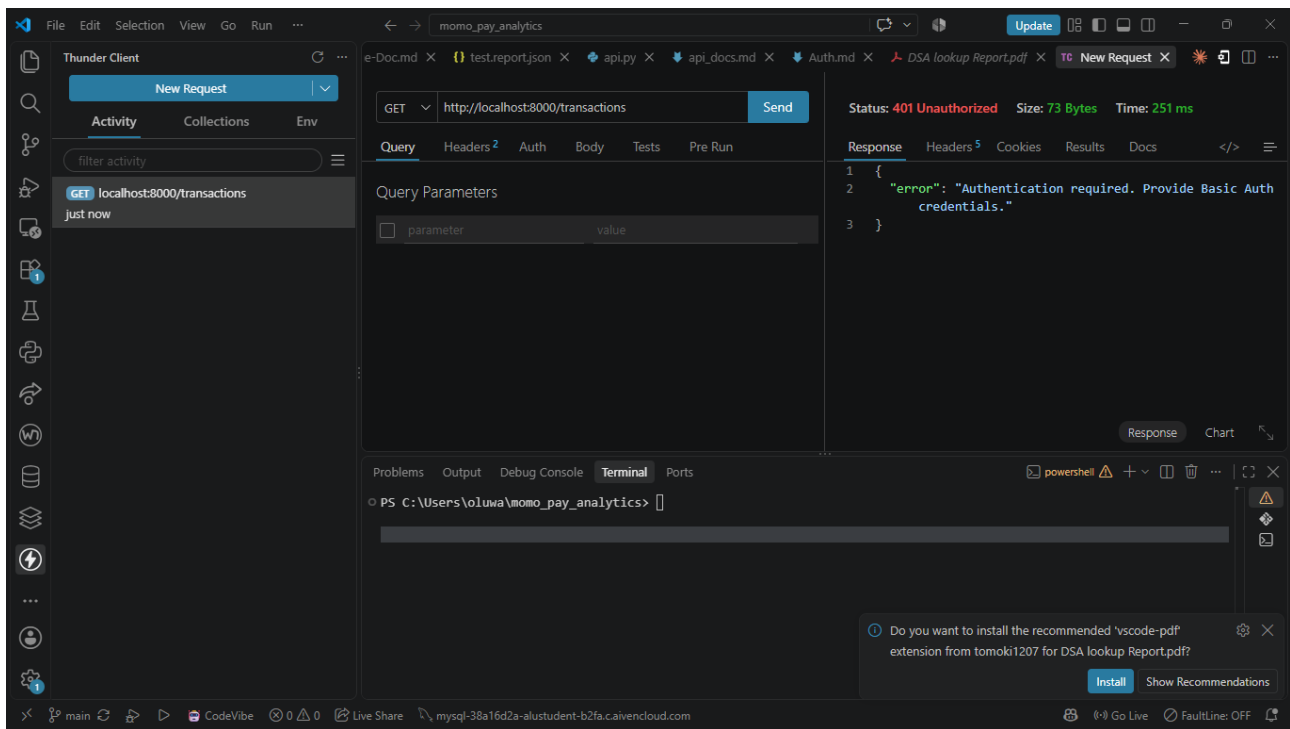


MoMo SMS REST API Testing Report

This report documents the testing of the MoMo SMS REST API endpoints using Thunder Client in Visual Studio Code. The API is built with Python's http.server module and uses HTTP Basic Authentication. The server was running on localhost:8000 with 1683 transactions loaded from data/transactions.json.

Test 1: GET /transactions without authentication



A GET request was sent to /transactions with no credentials. The server returned 401 Unauthorized with the message "Authentication required. Provide Basic Auth credentials." This confirms authentication is enforced on all endpoints.

Test 2: GET /transactions with Basic Auth

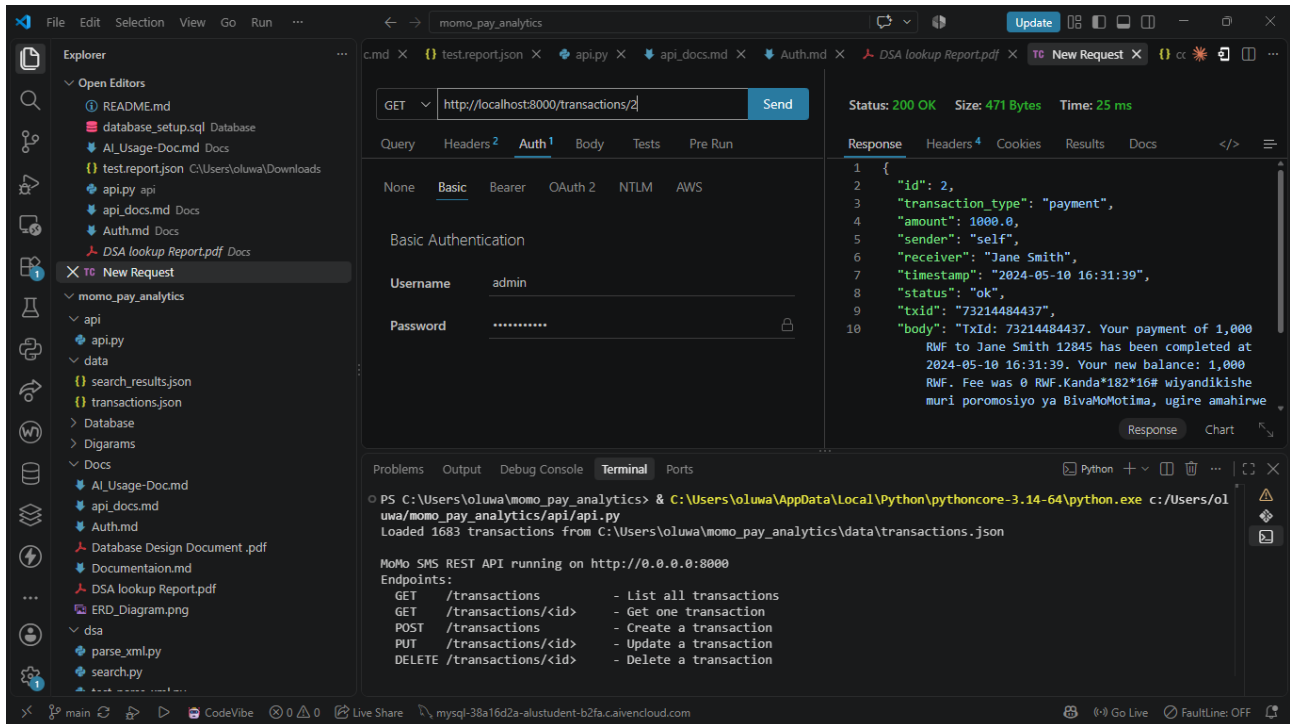
The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: GET
 - URL: `http://localhost:8000/transactions`
 - Auth: Basic Authentication
 - Username: `admin`
 - Password: `*****`
- Response:**
 - Status: 200 OK
 - Size: 746.51 KB
 - Time: 68 ms
 - Body (JSON):

```
1 {
2   "count": 1683,
3   "transactions": [
4     {
5       "id": 1,
6       "transaction_type": "received",
7       "amount": 2000.0,
8       "sender": "Jane Smith",
9       "receiver": "self",
10      "timestamp": "2024-05-10 16:30:51",
11      "status": "ok",
12      "txid": "76662021700",
13      "body": "You have received 2000 RWF from Jane Smith (*****013
              ) on your mobile money account at 2024-05-10 16:30:51.
              Message from sender: . Your new balance:2000 RWF. Financial
              Transaction Id: 76662021700."
14     },
15     {
16       "id": 2,
17       "transaction_type": "payment",
18       "amount": 1000.0,
19       "sender": "self",
20       "receiver": "Jane Smith",
21       "timestamp": "2024-05-10 16:31:39",
22       "status": "ok",
23       "txid": "73214484437",
24       "body": "TxId: 73214484437. Your payment of 1,000 RWF to Jane
              Smith 12845 has been completed at 2024-05-10 16:31:39. Your
```

A GET request was sent with valid Basic Auth credentials (admin). The server returned 200 OK with all 1683 transactions. Each transaction includes id, transaction_type, amount, sender, receiver, timestamp, status, txid, and body.

Test 3: GET /transactions/2 (single transaction)



The screenshot shows the VS Code interface with the REST client extension. A GET request is sent to `http://localhost:8000/transactions/2`. The response is 200 OK with a status of 200 OK, size of 471 Bytes, and time of 25 ms. The response body is a JSON object:

```
{
  "id": 2,
  "transaction_type": "payment",
  "amount": 1000.0,
  "sender": "self",
  "receiver": "Jane Smith",
  "timestamp": "2024-05-10 16:31:39",
  "status": "ok",
  "txid": "73214484437",
  "body": "Txid: 73214484437. Your payment of 1,000 RWF to Jane Smith 12845 has been completed at 2024-05-10 16:31:39. Your new balance: 1,000 RWF. Fee was 0 RWF. Kanda*182*16# miyandikishe muri poromosiyo ya BivaMoMotima, ugire amahirwe"
}
```

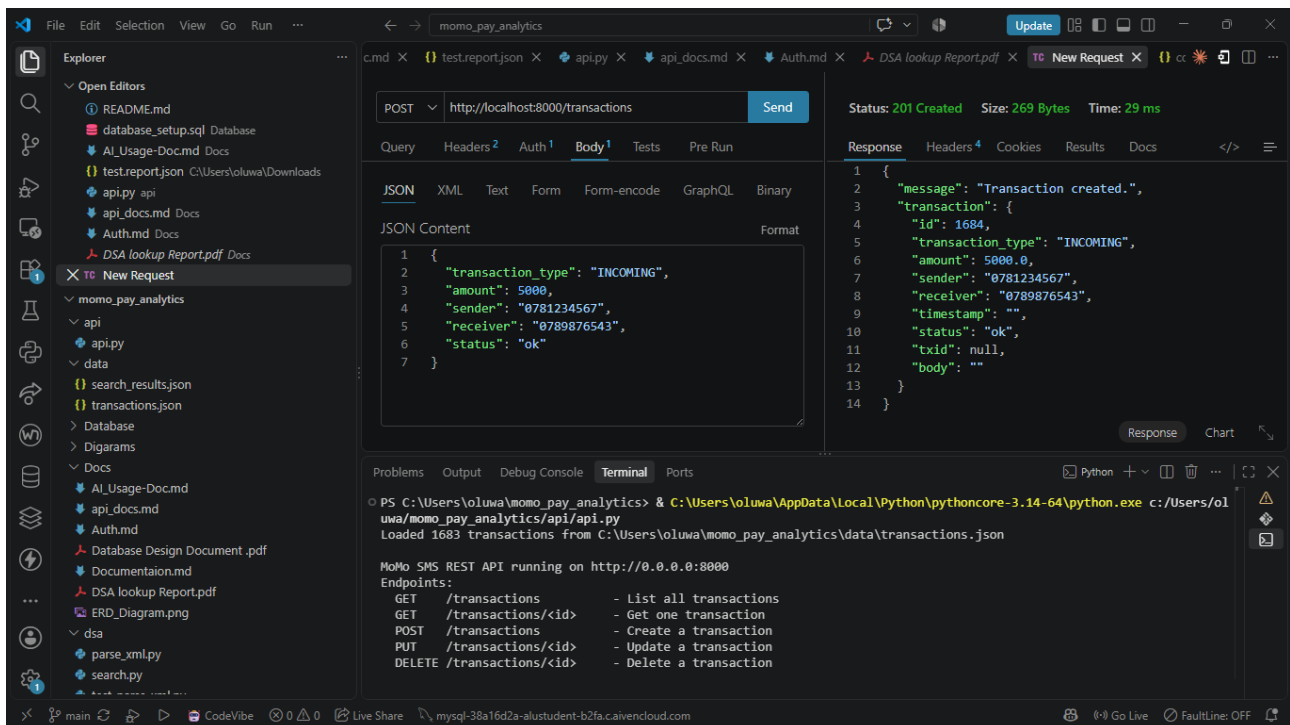
The terminal shows the command to run the API:

```
PS C:\Users\oluwa\momo_pay_analytics> & C:\Users\oluwa\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/oluwa/momo_pay_analytics/api/api.py
Loaded 1683 transactions from C:\Users\oluwa\momo_pay_analytics\data\transactions.json

MoMo SMS REST API running on http://0.0.0.0:8000
Endpoints:
GET /transactions - List all transactions
GET /transactions/<id> - Get one transaction
POST /transactions - Create a transaction
PUT /transactions/<id> - Update a transaction
DELETE /transactions/<id> - Delete a transaction
```

A GET request was sent to retrieve transaction ID 2. The server returned 200 OK with the full transaction object, a payment of 1,000 RWF to Jane Smith.

Test 4: POST /transactions (create)



The screenshot shows the VS Code interface with the REST client extension. A POST request is sent to `http://localhost:8000/transactions` with a JSON body:

```
{
  "transaction_type": "INCOMING",
  "amount": 5000,
  "sender": "0781234567",
  "receiver": "0789876543",
  "status": "ok"
}
```

The response is 201 Created with a status of 201 Created, size of 269 Bytes, and time of 29 ms. The response body is a JSON object:

```
{
  "message": "Transaction created.",
  "transaction": {
    "id": 1684,
    "transaction_type": "INCOMING",
    "amount": 5000.0,
    "sender": "0781234567",
    "receiver": "0789876543",
    "timestamp": "",
    "status": "ok",
    "txid": null,
    "body": ""
  }
}
```

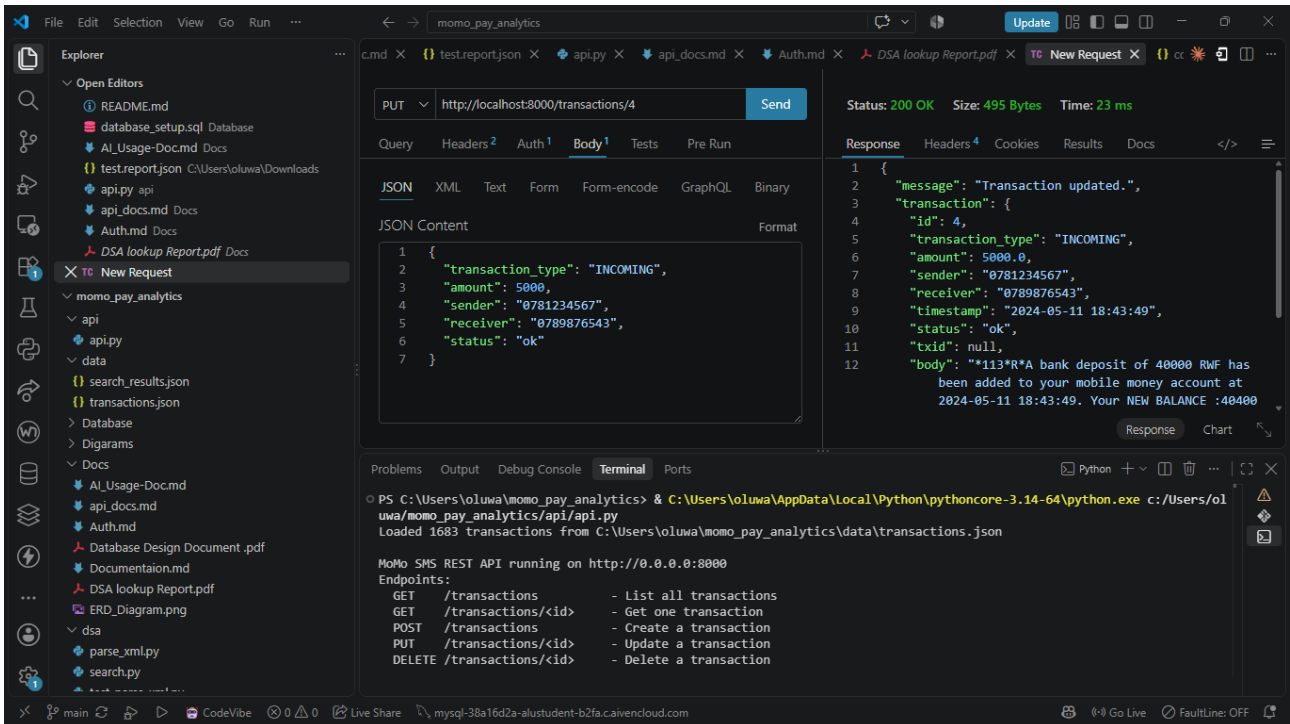
The terminal shows the command to run the API:

```
PS C:\Users\oluwa\momo_pay_analytics> & C:\Users\oluwa\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/oluwa/momo_pay_analytics/api/api.py
Loaded 1683 transactions from C:\Users\oluwa\momo_pay_analytics\data\transactions.json

MoMo SMS REST API running on http://0.0.0.0:8000
Endpoints:
GET /transactions - List all transactions
GET /transactions/<id> - Get one transaction
POST /transactions - Create a transaction
PUT /transactions/<id> - Update a transaction
DELETE /transactions/<id> - Delete a transaction
```

A POST request was sent with a JSON body containing transaction_type, amount, sender, receiver, and status. The server returned 201 Created with the new transaction assigned ID 1684.

Test 5: PUT /transactions/4 (update)



The screenshot shows the VS Code interface with the REST Client extension. A PUT request is being sent to `http://localhost:8000/transactions/4`. The request body is a JSON object:

```
1 {
2   "transaction_type": "INCOMING",
3   "amount": 5000,
4   "sender": "0781234567",
5   "receiver": "0789876543",
6   "status": "ok"
7 }
```

The response is 200 OK with a status of 200 OK, size of 495 Bytes, and time of 23 ms. The response body is a JSON object:

```
1 {
2   "message": "Transaction updated.",
3   "transaction": {
4     "id": 4,
5     "transaction_type": "INCOMING",
6     "amount": 5000.0,
7     "sender": "0781234567",
8     "receiver": "0789876543",
9     "timestamp": "2024-05-11 18:43:49",
10    "status": "ok",
11    "txid": null,
12    "body": "**113*R*A bank deposit of 40000 RWF has been added to your mobile money account at 2024-05-11 18:43:49. Your NEW BALANCE :40400"
13  }
14 }
```

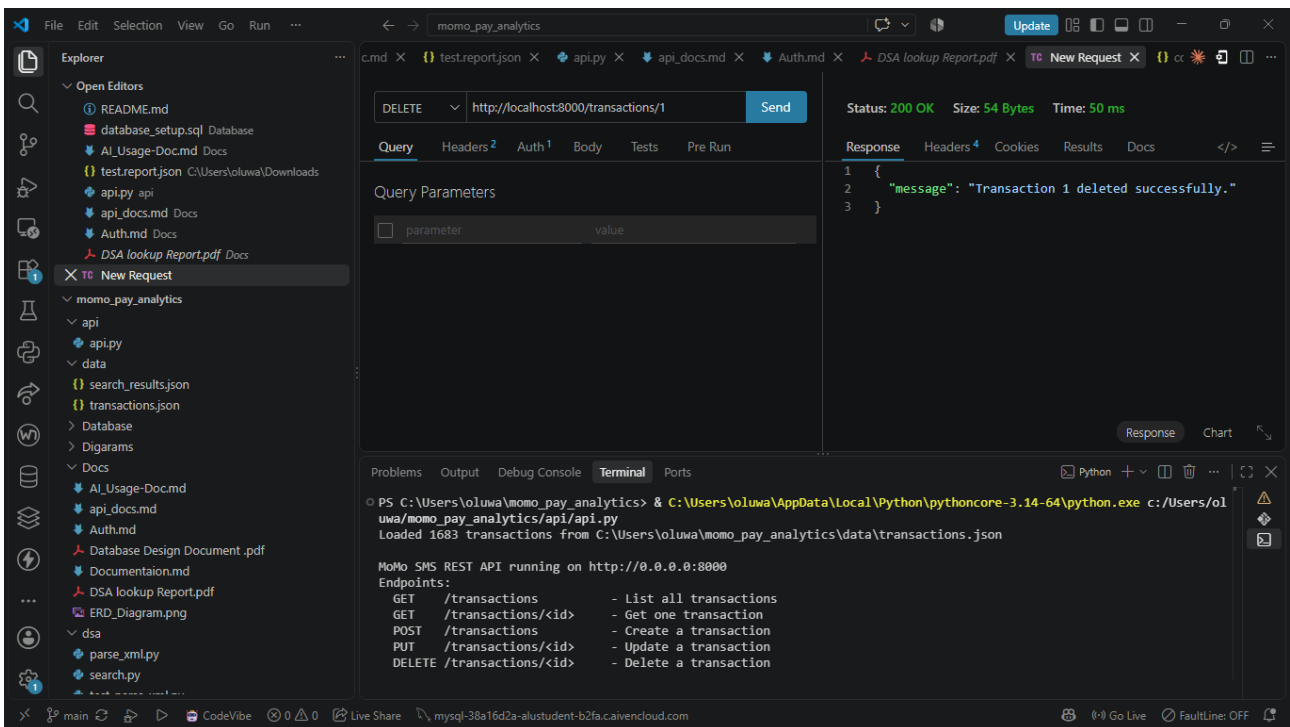
The terminal shows the command to run the API:

```
PS C:\Users\oluwa\momo_pay_analytics> & C:\Users\oluwa\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/oluwa/momo_pay_analytics/api/api.py
Loaded 1683 transactions from C:\Users\oluwa\momo_pay_analytics\data\transactions.json

MoMo SMS REST API running on http://0.0.0.0:8000
Endpoints:
GET /transactions - List all transactions
GET /transactions/<id> - Get one transaction
POST /transactions - Create a transaction
PUT /transactions/<id> - Update a transaction
DELETE /transactions/<id> - Delete a transaction
```

A PUT request was sent to update transaction ID 4 with new values. The server returned 200 OK with the updated transaction. Only the provided fields were changed.

Test 6: DELETE /transactions/1



The screenshot shows the VS Code interface with the REST Client extension. A DELETE request is being sent to `http://localhost:8000/transactions/1`. The response is 200 OK with a status of 200 OK, size of 54 Bytes, and time of 50 ms. The response body is a JSON object:

```
1 {
2   "message": "Transaction 1 deleted successfully."
3 }
```

The terminal shows the command to run the API:

```
PS C:\Users\oluwa\momo_pay_analytics> & C:\Users\oluwa\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/oluwa/momo_pay_analytics/api/api.py
Loaded 1683 transactions from C:\Users\oluwa\momo_pay_analytics\data\transactions.json

MoMo SMS REST API running on http://0.0.0.0:8000
Endpoints:
GET /transactions - List all transactions
GET /transactions/<id> - Get one transaction
POST /transactions - Create a transaction
PUT /transactions/<id> - Update a transaction
DELETE /transactions/<id> - Delete a transaction
```

A DELETE request was sent to remove transaction ID 1. The server returned 200 OK with the message "Transaction 1 deleted successfully."

Conclusion

All six tests passed. The API correctly enforces authentication, returns proper status codes, and supports all CRUD operations: listing and retrieving transactions (GET), creating new transactions (POST), updating existing ones (PUT), and deleting by ID (DELETE).